

Auto-Run Overview — What the System Does on Its Own

A guided map of everything that runs without an explicit user action, and exactly what keeps happening when the server is up but nobody has a browser tab open. Everything below is drawn directly from the code in this repository — real function names, intervals, and env knobs — not assumptions about how a dashboard "usually" behaves.

The one line that frames everything. With the server up and **no user active**, exactly one thing keeps running: a single daemon thread that **syncs prices and evaluates alerts every 5 minutes** (and refreshes analyst targets roughly hourly by riding on that same loop). Everything else is **browser-driven** — it only runs while a tab is open — and every server-side cache is **passive**: it recomputes lazily on the next request after it expires, never on a timer.

There are three independent "who triggers it" layers, and conflating them is the usual source of confusion:

```
+-----+
| SERVER-DRIVEN (runs with no browser open) |
|   background_sync_loop() — main.py       |
|   daemon Thread: while True -> sync_prices + evaluate_all |
|   every 300s; analyst targets every ~12th cycle (~1h)    |
+-----+
|                                     | (the ONLY scheduler)
|                                     v
+-----+
| CLIENT-DRIVEN (stops the moment the tab closes) |
|   setInterval 30s view refresh (dashboard.html) |
|   top-news feed throttled to <= 5 min           |
|   per-symbol news sentiment on selectSymbol() (memoized) |
+-----+
|                                     |
|                                     v
+-----+
| PASSIVE CACHES (never self-refresh) |
|   TtlCache.get(key, factory) across services/* |
|   recompute only on a request after TTL expiry |
+-----+
```

Section 1 — Server-driven auto-runs

Where: `main.py` (`background_sync_loop`, `ensure_background_worker`, `start_server`).

The only scheduled job in the system

A single background **daemon threading.Thread** runs `background_sync_loop()`, which is a plain `while True:` loop ending in `time.sleep(300)` — so it fires **every 5 minutes** for as long as the process lives. On each cycle, if the portfolio has any symbols, it:

1. `portfolio_service.sync_prices(get_engine(), refresh_targets=...)` — refreshes prices for every tracked symbol.

2. `alerts_service.evaluate_all(get_engine())` — re-evaluates all alert rules and logs any newly-triggered alerts.

Every **12th** cycle (~1 hour), it *also* refreshes analyst targets by passing `refresh_targets=True`. The cadence is the env var `TARGET_REFRESH_CYCLES` (default **12**), floored at 1:

```
```131:159:main.py def background_sync_loop(): """Background worker that continuously syncs prices via the
engine.""" set_current_user_id(get_bootstrap_user_id()) cycle = 0 target_refresh_every = max(1,
int(os.environ.get("TARGET_REFRESH_CYCLES", "12")),) while True: symbols = portfolio_service.list_symbols()
if symbols: cycle += 1 refresh_targets = cycle % target_refresh_every == 0 # ... sync_prices(...) then
alerts_service.evaluate_all(...) ... time.sleep(300)
```

```
> **Note on the cycle counter.** `cycle` only increments on cycles where the
> portfolio is non-empty, so "every 12th cycle" means every 12th *populated*
> cycle. With an empty portfolio the loop still spins every 5 minutes but does no
> work.
```

### ### How and when it starts

The worker is started **\*\*exactly once\*\*** via ``ensure_background_worker()``, guarded by a lock + a ``_sync_worker_started`` flag, so it can never double-start:

```
- **Boot path (dev / `python main.py`):** `start_server()` calls
`ensure_background_worker()` directly, so the thread is alive before the first
request.
- **WSGI / gunicorn path:** there is no boot hook, so it starts **lazily on the
first HTTP request** via `@app.before_request` → ensure_database()`, which calls
`ensure_background_worker()`.
```

Once started it runs **\*\*forever, regardless of users\*\*** — the ``before_request`` trigger is only how it gets *\*kick-started\** under gunicorn; nothing about it is per-user or per-request after that.

### ### What is *\*not\** here

There is **\*\*no\*\*** cron, **\*\*no\*\*** APScheduler / ``BackgroundScheduler`` / ``add_job``, no Flask ``before_first_request``, and no signal-based timer (``signal.alarm``) **\*\*anywhere\*\*** in the codebase. This one thread is the entirety of server-side scheduling. (Verified: a repo-wide search for those constructs returns nothing.)

---

### ## Section 2 — Client-driven auto-runs

**\*\*Where:\*\*** ``dashboard.html`` (browser JavaScript). All of these are tied to a live tab and **\*\*stop** the instant the tab is closed or the process serving it ends.\*\*

### ### View auto-refresh — every 30s

``startAutoRefresh()`` registers a single ``setInterval(autoRefreshTick, AUTO_REFRESH_MS)`` with ``AUTO_REFRESH_MS = 30000``, so the **\*\*current view\*\*** (Overview / Holdings, via ``autoRefreshLoaders[currentView]``) silently reloads every **\*\*30 seconds\*\***. It is deliberately conservative — ``autoRefreshTick()`` self-suppresses when any of these is true:

```
- `document.hidden` — the tab is backgrounded;
- `autoRefreshInProgress` — a refresh is already in flight;
- `userIsInteracting()` — the user is typing in an `INPUT` / `TEXTAREA` /
```

```

contenteditable.
```3567:3587:dashboard.html
  async function autoRefreshTick() {
    // Skip when the tab is hidden, a refresh is mid-flight, or the user is
    // typing – so we never clobber input or waste calls in the background.
    if (document.hidden || autoRefreshInProgress || userIsInteracting()) return;
    const loader = autoRefreshLoaders[currentView];
    if (!loader) return;
    autoRefreshInProgress = true;
    try {
      await loader();
      markRefreshed(true);
    } catch (_) {
      /* background refresh failures are non-fatal and stay silent */
    } finally {
      autoRefreshInProgress = false;
    }
  }

  function startAutoRefresh() {
    if (autoRefreshTimer) return;
    autoRefreshTimer = setInterval(autoRefreshTick, AUTO_REFRESH_MS);
  }

```

Top-news / recommendation-changes feed — at most every 5 min

The news feed is the only Yahoo-heavy piece on the Overview, so it is **throttled independently** of the 30s tick. Inside the Overview render, `loadTopNews()` runs only if `Date.now() - lastTopNewsAt >= TOPNEWS_REFRESH_MS`, with `TOPNEWS_REFRESH_MS = 300000` (**5 minutes**). The first render always runs since `lastTopNewsAt` starts at 0. So even though the view refreshes every 30s, news is re-pulled at most once every 5 minutes — and **only while the browser is refreshing Overview**.

```

```5073:5077:dashboard.html
const nowTs = Date.now();
if (nowTs - lastTopNewsAt >= TOPNEWS_REFRESH_MS) {
 lastTopNewsAt = nowTs;
 loadTopNews();
}

```

### Per-symbol news sentiment – on selection, memoized

When the user opens a symbol, `selectSymbol()` calls `resolveNewsSentiment()` (the price-reaction event study). This is **not** on a timer – it fires on the selection event – and it is **session-memoized** in a `Map` (`newsSentimentCache`) keyed by uppercased symbol, so it never re-runs for a symbol already resolved in that session:

```

```5642:5658:dashboard.html
function resolveNewsSentiment(symbol) {
  const key = String(symbol || "").toUpperCase();
  if (!key) return Promise.resolve(null);
  if (newsSentimentCache.has(key)) {
    return Promise.resolve(newsSentimentCache.get(key));
  }
  return api(`/symbols/${encodeURIComponent(key)}/news-sentiment`)
    .then((res) => {
      const ns = res ? res.newsSentiment || null : null;
      newsSentimentCache.set(key, ns);
      return ns;
    })
    .catch(() => {

```

```
        newsSentimentCache.set(key, null);
        return null;
    });
}
```

Other render-time loaders — event-driven, no timers

The remaining loaders (the Inspector, fundamentals, latest assessments, track record, and news-reaction badges) are **render-time / event-driven**: they run when their view or symbol is opened, not on any independent timer. For example, `loadLatestAssessments()` and `loadTrackRecord()` run as part of the Overview render, alongside the throttled news pull.

Section 3 — Never auto-runs (explicit user action only)

Two of the most expensive operations are **never** scheduled and never fire on any timer — they happen **only** when the user clicks the corresponding button:

Operation	Trigger	Endpoint
Portfolio / symbol assessment	"Assess" button	POST <code>/api/v1/assess</code> , POST <code>/api/v1/symbols/<symbol>/assess</code>
Note synthesis	"Synthesize" button	POST <code>/api/v1/notes/synthesize</code>

(The companion doc *Assessment, Recommendation & News* states the same: "Assessments are on-demand, never scheduled.") The background loop deliberately does **not** call the LLM — it only syncs prices, evaluates alerts, and periodically refreshes analyst targets.

Section 4 — Server-side caches (clarification)

A reasonable worry is that the various caches might "wake up" and recompute on their own. **They do not.** Every cache in `services/` is an instance of `TtlCache` (`services/market_cache.py`) accessed through the lazy `TtlCache.get(key, factory)` pattern: the factory runs **only** when a request asks for a key that is missing or expired. Nothing iterates the caches on a timer; nothing pre-warms them in the background.

Cache	File	Refresh model
<code>_PRICE_CACHE</code> , <code>_INTRADAY_CACHE</code>	<code>services/news_relevance_service.py</code>	lazy on request after TTL
<code>news_cache</code> , <code>finnhub_fundamentals_cache</code> , <code>analyst_targets_cache</code> , <code>yf_failure_cache</code>	<code>services/fundamentals_service.py</code>	lazy on request after TTL

<code>ticker_info_cache</code>	<code>services/market_cache.py</code> (used in <code>inspector_service.py</code> , <code>fundamentals_service.py</code>)	lazy on request after TTL
<code>_YTD_PRICE_CACHE</code>	<code>services/overview_service.py</code>	lazy on request after TTL
<code>_history_cache</code> , <code>_history_fail_cache</code>	<code>services/technical_signals_service.py</code>	lazy on request after TTL
fib <code>_CACHE</code>	<code>services/fib_service.py</code>	lazy on request after TTL

The other threading uses are not schedulers. Everything else thread-shaped in `services/` is a **concurrency primitive**, not a timer: `ThreadPoolExecutor` fan-out for per-request work (`market_cache.get_pool`, `assessment_service`'s parallel LLM calls, the Patterns view) and `Lock`/`Semaphore` guards. They only run while a request is being served, and they exit when it finishes. The only `while True` loop in the entire codebase is `background_sync_loop()` in `main.py`.

Activity classification (at a glance)

Activity	Trigger	Cadence	Client vs. server	File / function
Price sync + alert evaluation	Background daemon thread	every 5 min	Server (no browser needed)	<code>main.py</code> · <code>background_sync_loop</code> → <code>portfolio_service.sync_prices</code> <code>alerts_service.evaluate_alerts</code>
Analyst-target refresh	Rides the same loop	~every 1 h (every 12th cycle)	Server	<code>main.py</code> · <code>background_sync_loop</code> (<code>TARGET_REFRESH_CYCLES</code>)
Worker startup	Boot, or first HTTP request under WSGI	once	Server	<code>main.py</code> · <code>ensure_background_worker</code> / <code>start_server</code> / <code>@app.before_request</code> <code>ensure_database</code>
View auto-refresh (Overview/Holdings)	<code>setInterval</code>	every 30 s (skips when hidden/typing/in-flight)	Client	<code>dashboard.html</code> · <code>autoRefreshTick</code> / <code>startAutoRefresh</code> (<code>AUTO_REFRESH_MS</code>)

Top-news / rec-changes feed	During Overview refresh, throttled	≤ every 5 min	Client	<code>dashboard.html</code> · <code>loadTopNews</code> (<code>TOPNEWS_REFRESH_MS</code>)
Per-symbol news sentiment	<code>selectSymbol()</code> (memoized)	on demand, once/symbol/session	Client	<code>dashboard.html</code> · <code>resolveNewsSentiment</code> (<code>newsSentimentCache</code> Map)
Inspector / fundamentals / assessments / track record / reaction badges	View or symbol open	event-driven	Client	<code>dashboard.html</code> render-time loaders
Portfolio / symbol assessment	"Assess" button	never auto	Client-initiated, server-executed	POST / <code>assess</code> , POST /symbols/<symbol>/ <code>assess</code>
Note synthesis	"Synthesize" button	never auto	Client-initiated, server-executed	POST /notes/ <code>synthesize</code>
Market / fundamentals / history / fib caches	Request after TTL expiry	lazy, passive	Server (no timer)	<code>TtlCache.get(key,</code> <code>factory)</code> across <code>services/*</code>

Bottom line

- **Server up, no user active:** only the **5-minute** price-sync + alert-evaluation thread runs, plus the **~hourly** analyst-target refresh that rides on it. That single daemon thread is the *entire* server-side schedule.
- **Everything else is browser-driven and dormant** without an open tab: the 30s view refresh, the ≤5-min news feed, and the on-select news sentiment all live in `dashboard.html` and stop when the tab closes.
- **Assessments and note synthesis never auto-run** — they are strictly button-triggered.
- **Caches are passive.** They recompute lazily on the next request after expiry; nothing refreshes them on a timer. Other threading usage is concurrency (pools, locks), not scheduling.

Implementation: `main.py` (`background_sync_loop`, `ensure_background_worker`, `start_server`), `dashboard.html` (`autoRefreshTick` / `startAutoRefresh`, `loadTopNews`, `resolveNewsSentiment`), and the `TtlCache`-backed caches in `services/` (`market_cache.py`, `news_relevance_service.py`, `fundamentals_service.py`, `overview_service.py`, `technical_signals_service.py`, `fib_service.py`).